Universidade de Brasília
Departamento de Ciência da Computação
Bacharelado em Ciência da Computação

# Geração de Dados Imediatos no RISC-V

**Aluno:** Lucas Vasconcelos Pirani Carneiro
**Matrícula:** 251031785

# 1 Introdução

O conjunto de instruções RISC-V utiliza diferentes formatos de instrução, que distribuem os campos de dados e controle em posições específicas de uma palavra de 32 bits. Em determinadas instruções, parte desses bits representa um valor imediato que deve ser extraído e, quando necessário, estendido para 32 bits.

Este trabalho consiste na implementação, em VHDL, do módulo `genImm32`, responsável por receber uma instrução RISC-V de 32 bits e gerar o imediato correspondente.

# 2 Objetivos

## 2.1 Objetivo Geral

Desenvolver um módulo em VHDL que gere os dados imediatos utilizados nas instruções do processador RISC-V [1].

## 2.2 Objetivos Específicos

- Identificar os formatos de instrução a partir do campo `opcode`;
- Extrair e reorganizar os bits dos imediatos dos formatos I, I*, S, SB, UJ e U;
- Verificar o funcionamento do módulo por meio de um testbench;
- Responder os questionamentos feitos na especificação.

# 3 Fundamentação Teórica

## 3.1 Formatos de Instrução

Cada formato define uma disposição própria dos campos que compõem a instrução. A identificação é feita pelo campo `opcode`, com exceção do formato Tipo-I* [1].
A tabela a seguir mostra como os formatos são identificados pelos `opcodes`:

| **Formato** | **Condição de Identificação** |
|---|---|
| R | `opcode = 0x33` |
| I | `opcode = 0x03, 0x13 ou 0x67` |
| I* | `opcode = 0x13`, `funct3 = 5` e `instr(30) = 1` |
| S | `opcode = 0x23` |
| SB | `opcode = 0x63` |
| U | `opcode = 0x37` |
| UJ | `opcode = 0x6F` |

## 3.2 Formação dos Imediatos

A unidade `ImmGen` extrai os bits do imediato conforme o formato da instrução e os reorganiza em um valor de 32 bits. Nos formatos I e S, o imediato possui 12 bits e é estendido com sinal; no formato I*, o campo `shamt` possui 5 bits e é completado com zeros. Nos formatos SB e UJ, o imediato representa um deslocamento par, portanto seu bit menos significativo é zero; no formato U, os 12 bits menos significativos são preenchidos com zeros [2].

Após a reorganização, os imediatos dos formatos I, S, SB e UJ são estendidos com sinal para 32 bits, replicando o bit de sinal presente em `inst[31]`. Essa posição fixa do bit de sinal simplifica o hardware, pois permite uma extensão de sinal uniforme para todos os formatos [2], [1].

| **Formato** | **Regra de Formação** | **Signed** |
|---|---|---|
| I | `imm(11:0) = instr(31:20)` | Sim |
| I* | `imm(4:0) = instr(24:20)` | Não |
| S | `imm(11:0) = instr(31:25) & instr(11:7)` | Sim |
| SB | `imm(12:1) = instr(31) & instr(7) & instr(30:25) & instr(11:8); imm(0) = 0` | Sim |
| UJ | `imm(20:1) = instr(31) & instr(19:12) & instr(20) & instr(30:21); imm(0) = 0` | Sim |
| U | `imm(31:12) = instr(31:12); imm(11:0) = 0` | Não |
| R | Não possui imediato; a saída deve ser zero. | Não se aplica |

# 4 Implementação

## 4.1 Interface

O módulo recebe a instrução pela entrada `instr`, do tipo `std_logic_vector(31 downto 0)`, e fornece o imediato pela saída `imm32`, do tipo `signed(31 downto 0)` [1].

```
entity genImm32 is
    port (
        instr : in  std_logic_vector(31 downto 0);
        imm32 : out signed(31 downto 0)
    );
end genImm32;
```

## 4.2 Arquitetura

A arquitetura `rtl` separa a identificação do formato da geração do imediato. Para representar os formatos, foi definido o tipo enumerado `FORMAT_RV`, cujos valores são `R_type`, `I_type`, `S_type`, `SB_type`, `UJ_type` e `U_type`.

O processo `p_formato` examina os bits do campo `opcode`. Os códigos `0000011`, `0010011` e `1100111` são associados ao formato I; os códigos `0100011`, `1100011`, `1101111` e `0110111` são associados, respectivamente, aos formatos S, SB, UJ e U. Os demais opcodes são encaminhados ao caso `R_type`.

Em seguida, o processo `p_gerador` seleciona a regra de formação do imediato conforme o formato identificado. Para as instruções do formato I, o campo `instr(31 downto 20)` é estendido com sinal. A instrução `srai` é tratada como caso especial: quando o opcode é `0010011`, `funct3` é `101` e `instr(30)` é igual a 1, o campo de cinco bits `instr(24 downto 20)` é estendido com zeros [1].

No formato S, o imediato é formado pela concatenação dos campos `instr(31 downto 25)` e `instr(11 downto 7)`, seguida de extensão de sinal. Nos formatos SB e UJ, os campos são reorganizados e o bit menos significativo do imediato é definido como zero antes da extensão de sinal. No formato U, os bits `instr(31 downto 12)` são concatenados com doze zeros. Para o caso `R_type`, que não possui imediato, o módulo atribui zero à saída `imm32` [1].

As conversões entre `signed` e `unsigned`, junto com a função `resize`, permitem realizar a extensão adequada e fornecer o resultado de 32 bits pela saída do módulo [1]. O código completo da arquitetura está disponível no arquivo `genImm32.vhd`.

# 5 Testes e Testbench

A verificação do módulo `genImm32` foi realizada utilizando as instruções de referência fornecidas na especificação do trabalho, contemplando os diferentes formatos de instrução RISC-V. Os casos de teste utilizados são apresentados na figura abaixo.

| Instrução RISC-V | Código | Formato | Imediato | Imm hex |
|---|---|---|---|---|
| add t0, zero, zero | 0x000002b3 | R-type | inexiste: 0 | 0x00000000 |
| lw t0, 16(zero) | 0x01002283 | I-type0 | 16 | 0x00000010 |
| addi t1, zero, -100 | 0xf9c00313 | I-type1 | -100 | 0xFFFFFF9C |
| xori t0, t0, -1 | 0xfff2c293 | I-type1 | -1 | 0xFFFFFFFF |
| addi t1, zero, 354 | 0x16200313 | I-type1 | 354 | 0x00000162 |
| jalr zero, zero, 0x18 | 0x01800067 | I-type2 | 24 | 0x00000018 |
| srai t1, t2, 10 | 0x40a3d313 | I-type* | 10 | 0x0000000A |
| lui s0, 2 | 0x00002437 | U-type | 0x2000 | 0x00002000 |
| sw t0, 60(s0) | 0x02542e23 | S-type | 60 | 0x0000003C |
| bne t0, t0, main | 0xfe5290e3 | SB-type | -32c | 0xFFFFFFE0 |
| jal rot | 0x00c000ef | UJ-type | 12 | 0x0000000C |

Para automatizar a verificação, foi desenvolvido um `testbench` em VHDL. Um testbench é um código utilizado para simular e verificar o comportamento de um módulo, aplicando entradas conhecidas e comparando as saídas obtidas com os valores esperados.

Neste trabalho, o `testbench` instancia o módulo `genImm32` e aplica, sequencialmente, os códigos das instruções à entrada `instr`. Após cada aplicação, aguarda-se um intervalo de 1

ns para a propagação dos sinais e, em seguida, verifica-se o valor de `imm32`. A comparação é realizada considerando os bits da saída, convertida de `signed` para `std_logic_vector`. Para isso, são utilizadas instruções `assert`, que sinalizam uma falha caso o resultado obtido seja diferente do esperado.

O código do testbench utilizado na verificação está disponível no arquivo `testbrench.vhd`.

## 5.1 Simulação

A simulação foi realizada na plataforma EDA Playground, que permite executar códigos de descrição de hardware utilizando diferentes simuladores. Para este trabalho, foi utilizado o `GHDL 5.1.1`.

Os resultados da simulação indicaram que todos os casos de teste produziram os valores esperados para os respectivos imediatos.

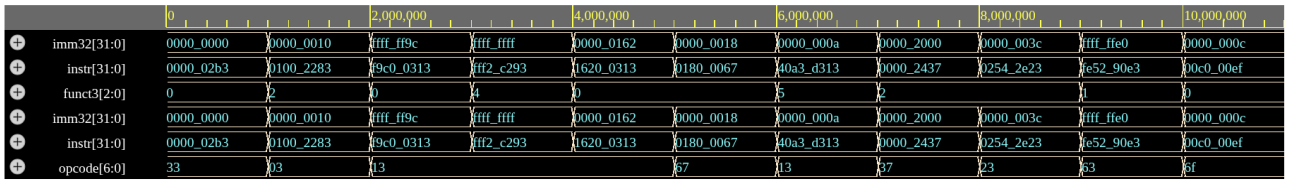



Figura 1: Simulação dos resultados em Hexadecimal

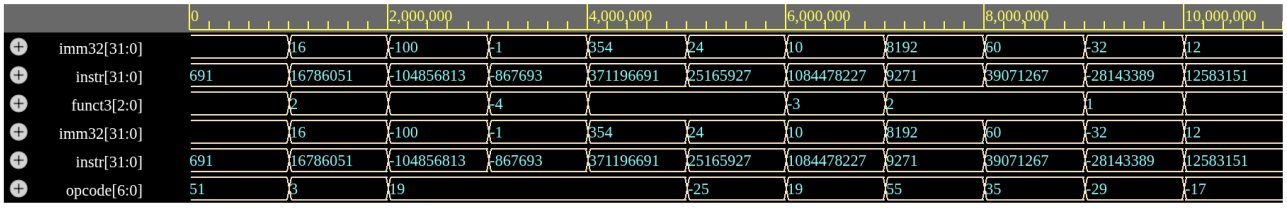



Figura 2: Simulação dos resultados em Signed Decimal

# 6 Discussão

## 6.1 Qual é a razão do embaralhamento dos bits do imediato no RISC-V?

O principal motivo para o embaralhamento dos bits dos imediatos em determinados formatos (S, SB, U e UJ) é para manter os registradores de origem `(rs1,rs2)` e destino `(rd)` exatamente nas mesmas posições de bits em todos os formatos de instrução [2].

## 6.2 Por que alguns imediatos não incluem o bit 0?

Nos formatos SB (desvios condicionais) e UJ (saltos incondicionais), o bit 0 do imediato é omitido porque os endereços de destino são alinhados em múltiplos de 2 bytes, considerando o suporte do RISC-V às instruções comprimidas de 16 bits. Dessa forma, o bit menos significativo

é sempre zero e pode ser definido implicitamente pelo hardware, sem necessidade de armazená-lo na instrução. Isso permite ampliar o alcance dos desvios e saltos sem aumentar o número de bits destinado ao imediato [2].

### 6.3 Os imediatos de operações lógicas estendem o sinal?

Sim. No RISC-V, todos os imediatos de 12 bits do formato I, como `andi, ori` e `xori`, passam obrigatoriamente por extensão de sinal [2].

### 6.4 Como é implementada a instrução NOT no RISC-V?

Como o RISC-V não possui uma instrução de hardware para o NOT, utiliza-se a instrução `xori rd, rs, -1` pois o imediato -1 em complemento de dois de 12 bits com extensão de sinal resulta em todos os bits iguais a 1, assim a operação de XOR bit a bit inverte os bits do registrador de origem já que `0 xor 1 = 1` e `1 xor 1 = 0` [2].

## 7 Conclusão

Este trabalho apresentou a implementação do módulo `genImm32` em VHDL, responsável por extrair e formar os dados imediatos das instruções RISC-V de 32 bits. O módulo identifica o formato da instrução por meio do campo `opcode` e realiza a extração, reorganização e extensão dos bits conforme os formatos I, I*, S, SB, U e UJ.

A verificação foi realizada por meio de um `testbench` com os casos de teste especificados, e a simulação no GHDL confirmou que todos os imediatos gerados corresponderam aos valores esperados. Dessa forma, o módulo implementado atende aos objetivos propostos e demonstra como a organização dos formatos de instrução do RISC-V facilita a decodificação e a extensão dos imediatos no hardware.

## Referências


[1] Especificação do trabalho: *Geração de Dados Imediatos no RISC-V*. Material fornecido pelo(a) professor(a), s.d.

[2] PATTERSON, David A.; HENNESSY, John L. *Computer Organization and Design RISC-V Edition: The Hardware/Software Interface*. 2. ed. Morgan Kaufmann, 2020. ISBN: 978-0-12-820331-6.